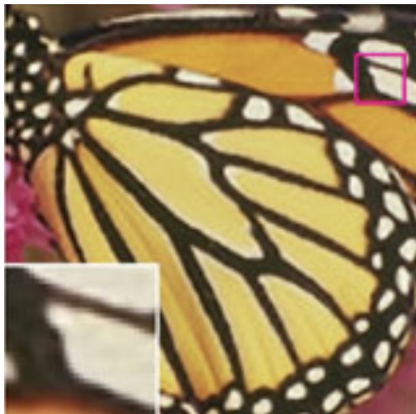
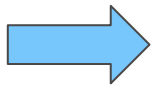
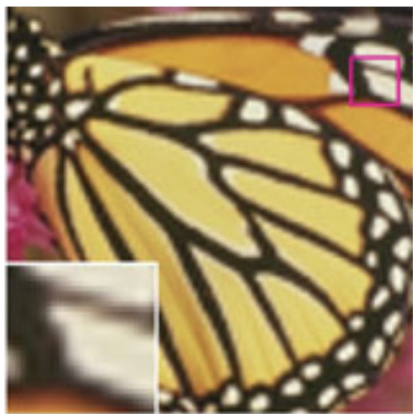
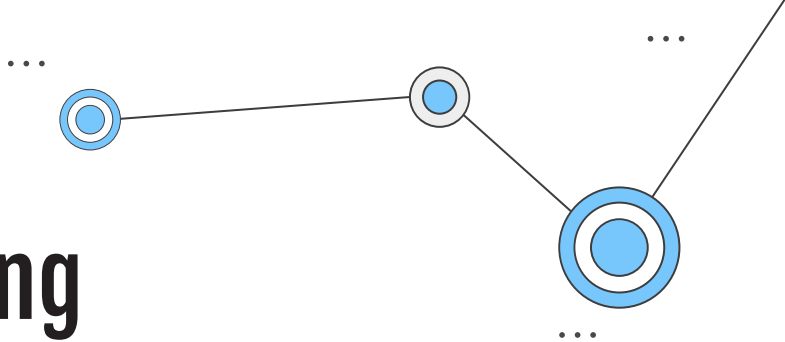


Image Super-Resolution Using Deep Convolutional Networks



Chaeun Ryu

Table of Contents



Introduction

...



Methodology

...



Experiments

...



Conclusion

...



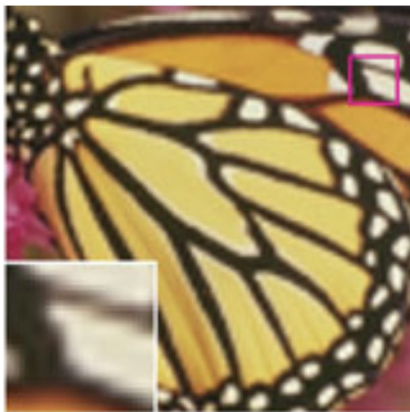
01

Introduction

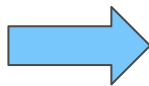


Super-Resolution (SR)

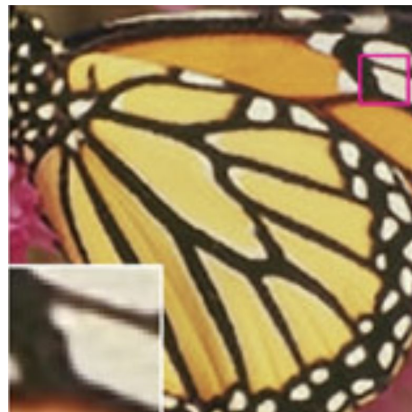
Aims to recover a high-resolution image from a single low-resolution image



Low Resolution Image



Super-Resolution
Methodology

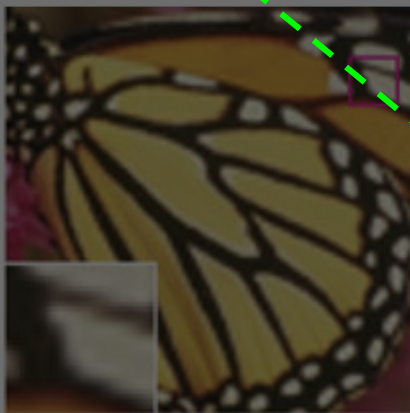


High Resolution Image

Introduction

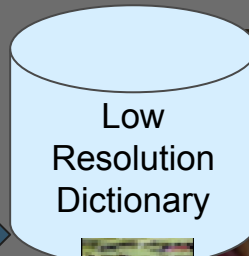
Previously...

The **Sparse-Coding-based method** was one of the most representative super-resolution method

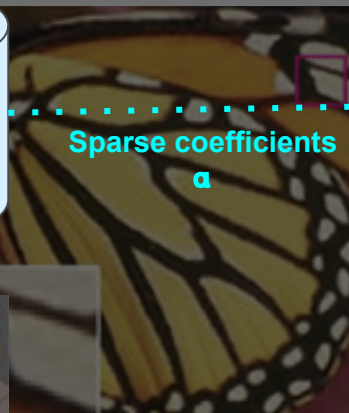


Low Resolution Image

Super-Resolution
Methodology



Low
Resolution
Dictionary



High Resolution Image



High
Resolution
Dictionary

*Dictionary $\sim$ vocabulary of basic patterns

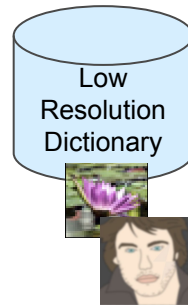
Sparse-Coding based method (brief overview)

01

Overlapping patches are densely cropped from the input image and processed (=normalized)

02

Patches are then encoded by low-resolution dictionary



03

The sparse coefficients α are passed into a high-resolution dictionary for reconstructing high resolution patches

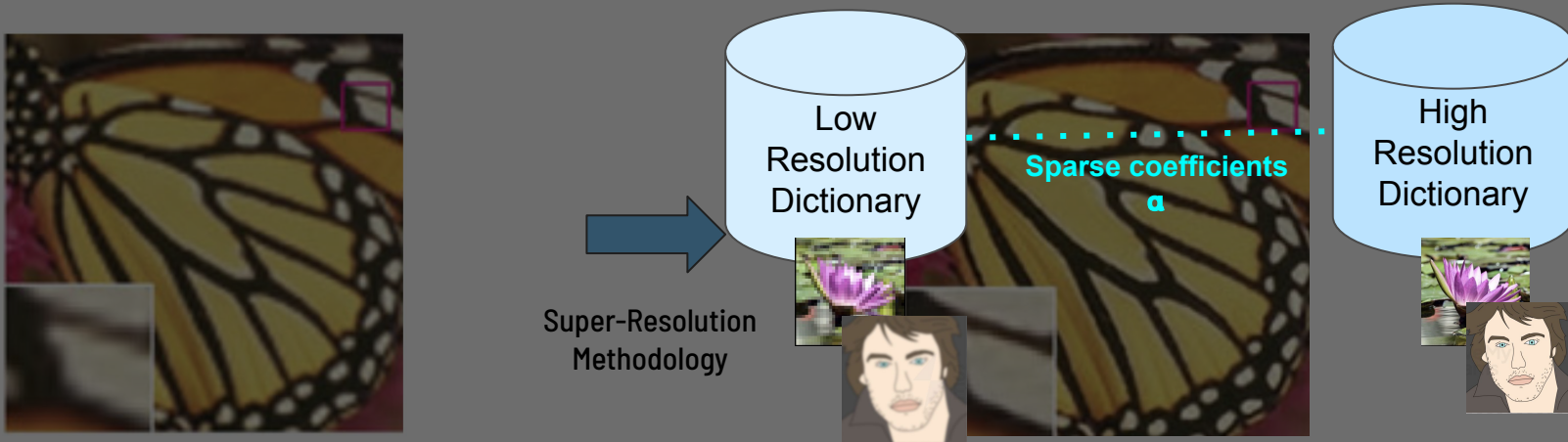
04

The overlapping reconstructed patches are aggregated (=weighted averaged) to produce the final output

Introduction

Previously...

The **Sparse-Coding-based method** was one of the most representative super-resolution method



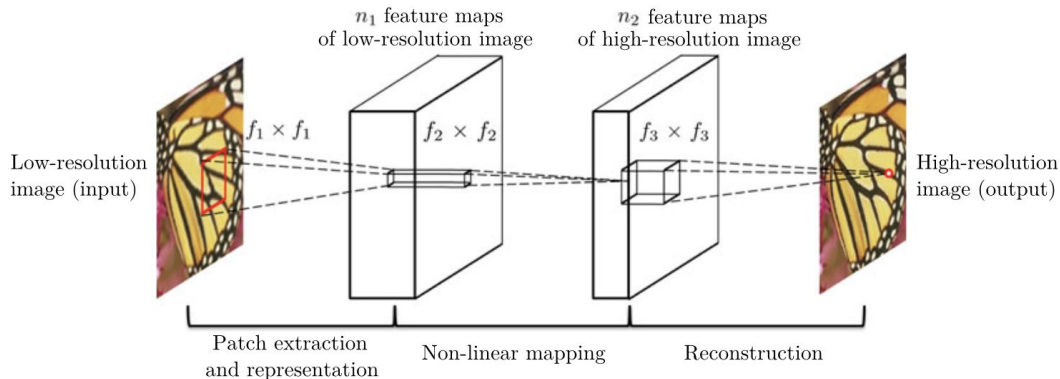
Here, the authors show that the SC method is equivalent to a **deep convolutional network**.

Introduction

Previously...

The **Sparse-Coding-based method** was one of the most representative super-resolution method

Super-resolution convolutional neural network (SRCNN)



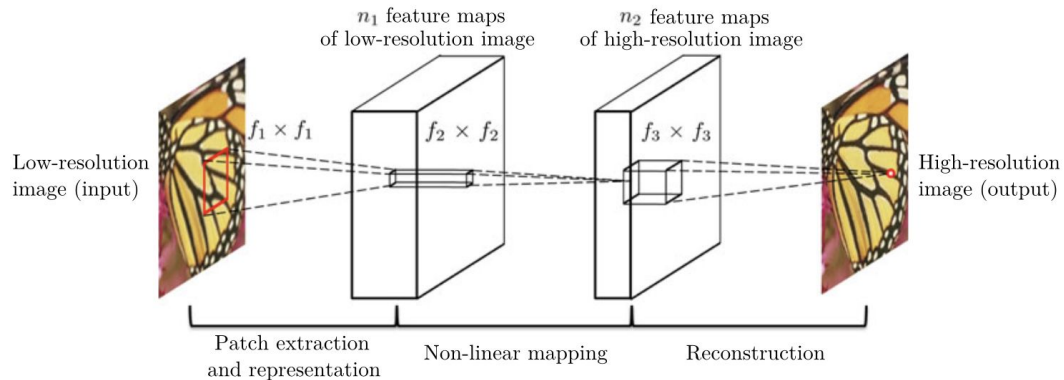
Here, the authors show that the SC method is equivalent to a deep convolutional network. Motivated by this fact, we consider a convolutional neural network that directly learns an end-to-end mapping between low- and high-resolution images.

Introduction

Previously...

The **Sparse-Coding-based method** was one of the most representative super-resolution method

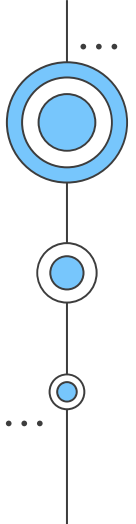
Super-resolution convolutional neural network (SRCNN)



- Simple
- Fast
- Superior performance

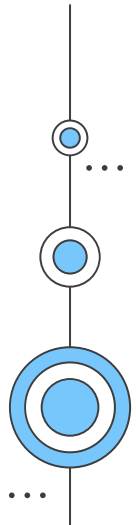
Low Resolution Image

High Resolution Image



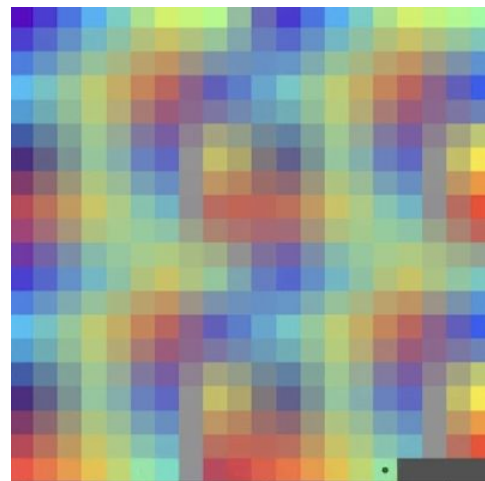
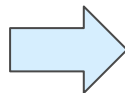
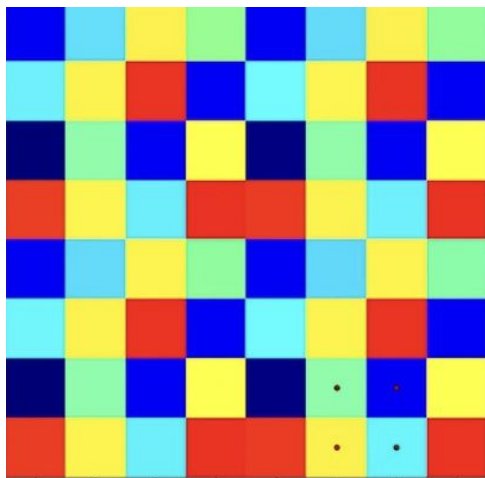
02

Methodology



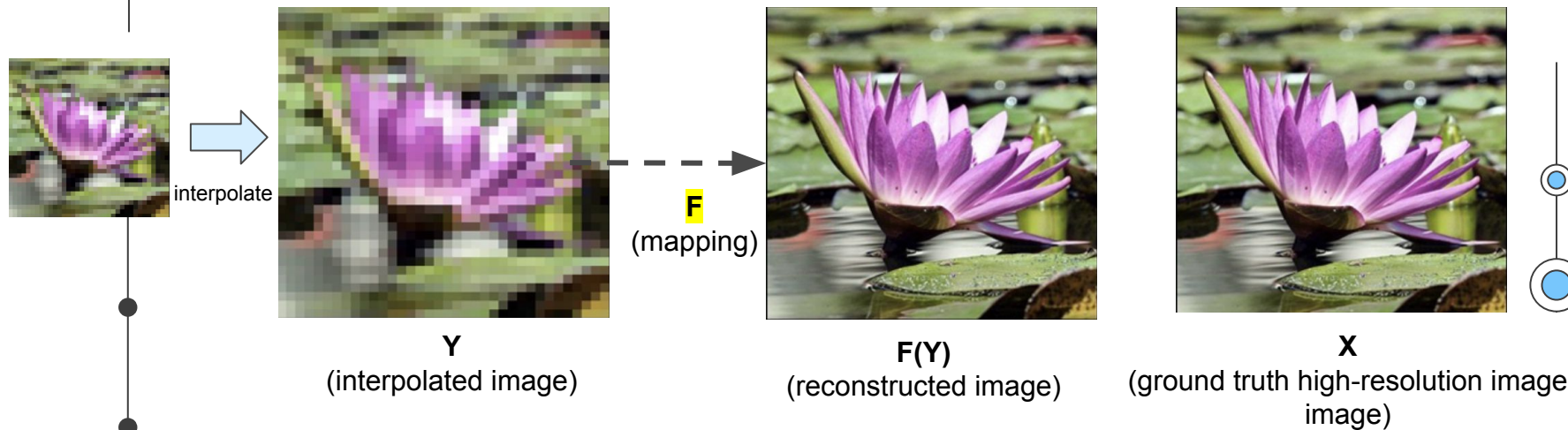
Formulation

Preprocessing step) Consider a single low-resolution image, we first upscale it to the desired size using bicubic interpolation

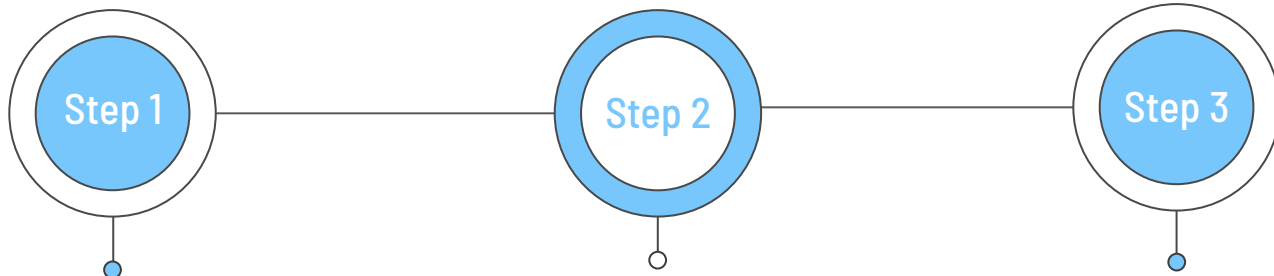


Formulation

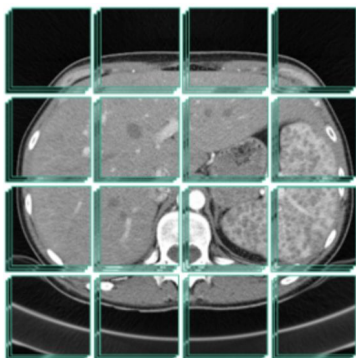
Our goal is to recover from $\mathbf{Y}$ an image $\mathbf{F}(\mathbf{Y})$ that is as similar as possible to the ground truth high-resolution image $\mathbf{X}$



Formulation

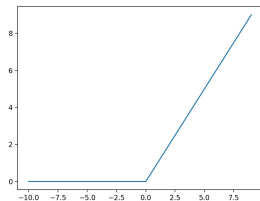


Patch Extraction and Representation



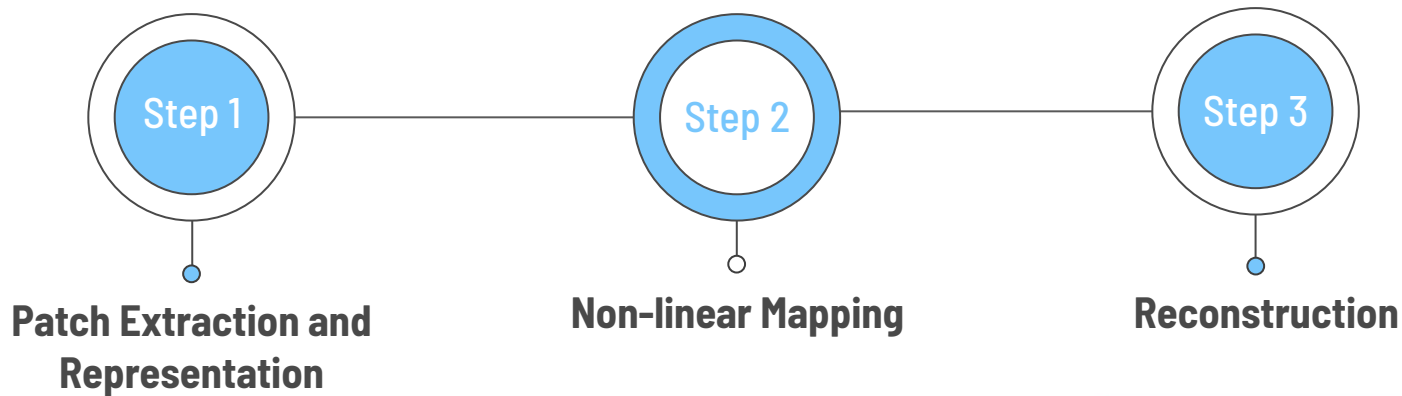
This operation **extracts (overlapping) patches** from the low-resolution image Y and **represents each patch as a high-dimensional vector**. These vectors comprise a set of feature maps, of which the number equals to the dimensionality of the vectors.

Formulation

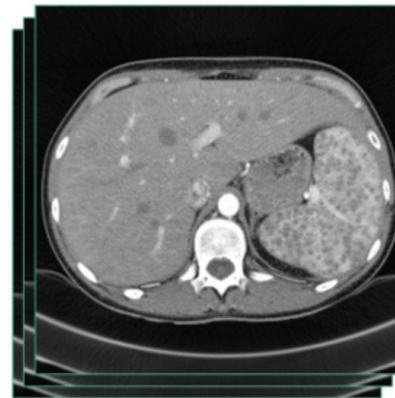


This operation **nonlinearly maps** each high-dimensional vector onto another high-dimensional vector. Each mapped vector is conceptually the representation of a high-resolution patch. These vectors comprise another set of feature maps.

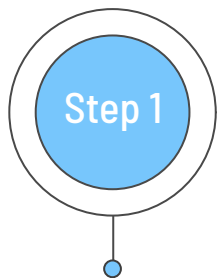
Formulation



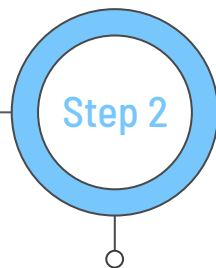
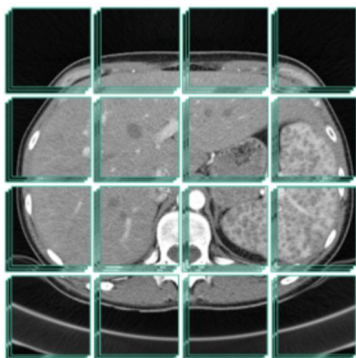
This operation **aggregates the above high-resolution patch-wise representations** to generate the final high-resolution image. This image is **expected to be similar to the ground truth X**.



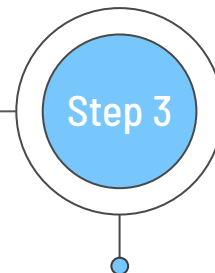
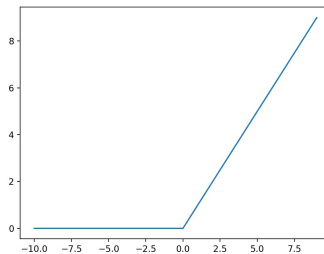
Formulation



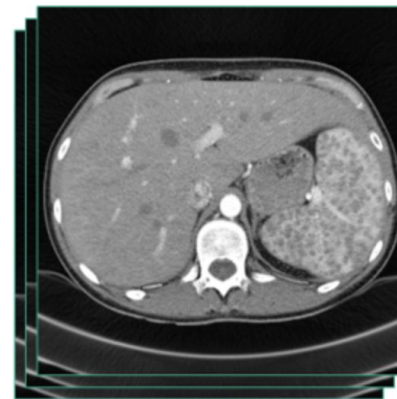
Patch Extraction and Representation



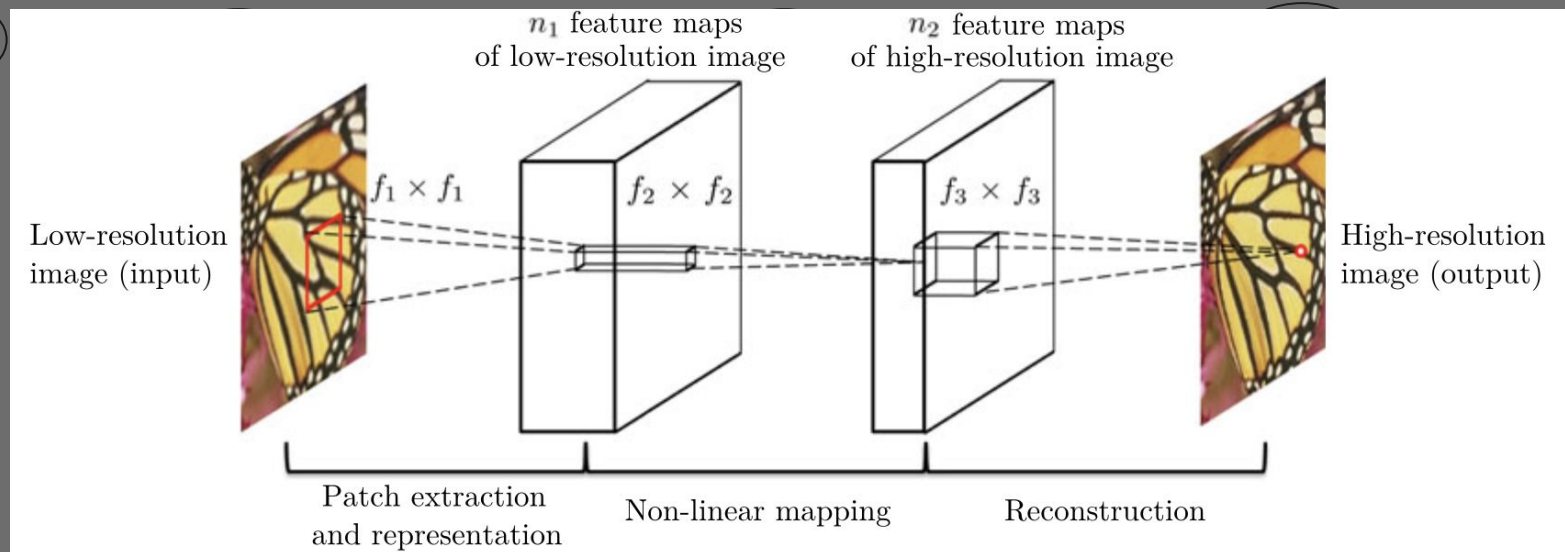
Non-linear Mapping



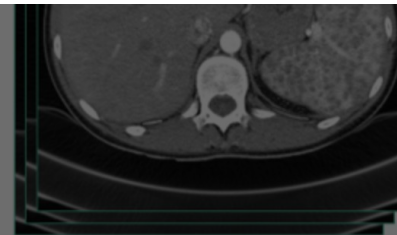
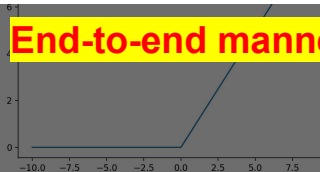
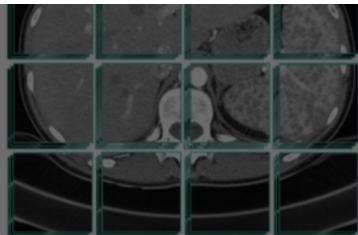
Reconstruction



Formulation



End-to-end manner!



1) Patch Extraction and Representation

$$F_1(\mathbf{Y}) = \max(0, W_1 * \mathbf{Y} + B_1)$$

- $\mathbf{Y}$: Low-resolution image
- W_1 : n_1 filters of support $c \times f_1 \times f_1$
- B_1 : n_1 - dimensional vector (bias)
- $*$: convolution operation
- $\max(0, x)$: ReLU operation (non-linearity)

1) Patch Extraction and Representation

$$F_1(\mathbf{Y}) = \max(0, \mathbf{W}_1 * \mathbf{Y} + \mathbf{B}_1)$$

- $\mathbf{W}_1$: n_1 filters of support $c \times f_1 \times f_1$
- $\mathbf{B}_1$: n_1 - dimensional vector (bias)
- $*$: convolution operation
- $\max(0, x)$: ReLU operation (non-linearity)

Applies n_1 convolutions on the image, each convolution has a kernel size of $c \times f_1 \times f_1$

Source layer

5	2	6	8	2	0	1	2
4	3	4	5	1	9	6	3
3	9	2	4	7	7	6	9
1	3	4	6	8	2	2	1
8	4	6	2	3	1	8	8
5	8	9	0	1	0	2	3
9	2	6	6	3	6	2	1
9	8	8	2	6	3	4	5

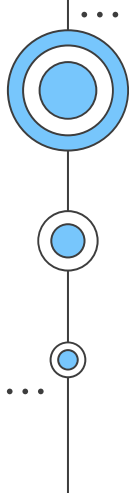
Convolutional kernel

-1	0	1
2	1	2
1	-2	0

Destination layer

		5					

$$\begin{aligned} & (-1 \times 5) + (0 \times 2) + (1 \times 6) + \\ & (2 \times 4) + (1 \times 3) + (2 \times 4) + \\ & (1 \times 3) + (-2 \times 9) + (0 \times 2) = 5 \end{aligned}$$



1) Patch Extraction and Representation

$$F_1(\mathbf{Y}) = \max(0, \mathbf{W}_1 * \mathbf{Y} + \mathbf{B}_1)$$

Applies n_1 convolutions on the image, each convolution has a kernel size of $c \times f_1 \times f_1$

- $\mathbf{W}_1$: n_1 filters of support $c \times f_1 \times f_1$
- $\mathbf{B}_1$: n_1 - dimensional vector (bias)
- $*$: convolution operation
- $\max(0, x)$: ReLU operation (non-linearity)

< Convolution as sliding patches >

- Convolution works by sliding each of these filters (n_1 of them in $\mathbf{W}_1$) across the input image $\mathbf{Y}$.
- At each spatial location of the image, the filter performs an element-wise multiplication with the corresponding $c \times f_1 \times f_1$ patch of the input image
- Then, it sums up all the values to produce a single output value. This output value is placed in the corresponding spatial location in the output feature map.

Source layer

5	2	6	8	2	0	1	2
4	3	4	5	1	9	6	3
3	9	2	4	7	7	6	9
1	3	4	6	8	2	2	1
8	4	6	2	3	1	8	8
5	8	9	0	1	0	2	3
9	2	6	6	3	6	2	1
9	8	8	2	6	3	4	5

Convolutional kernel

-1	0	1
2	1	2
1	-2	0

Destination layer

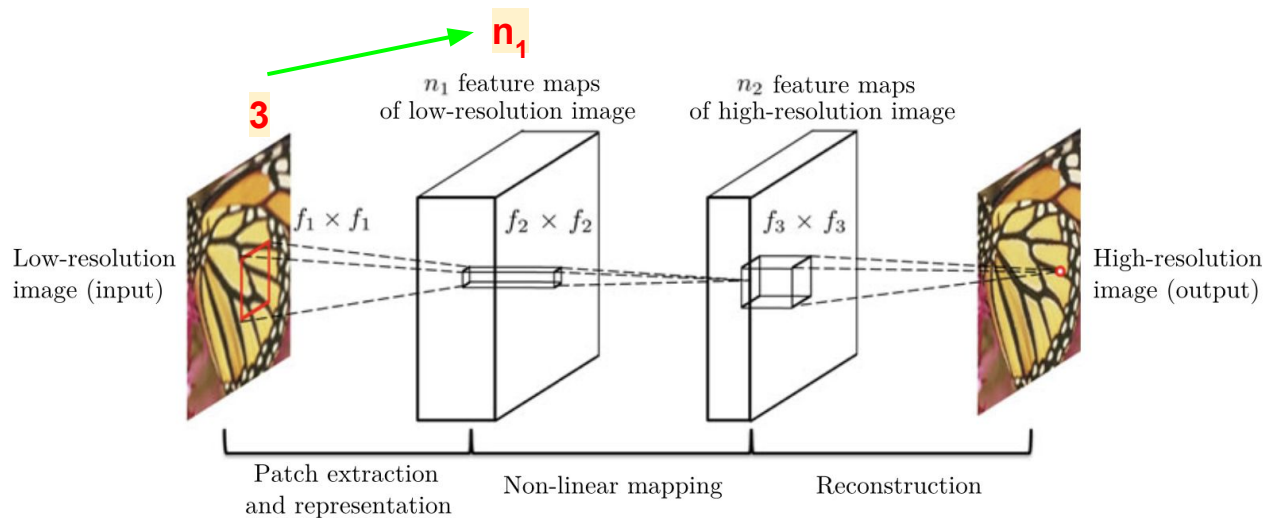
		5					

$$\begin{aligned}
 &(-1 \times 5) + (0 \times 2) + (1 \times 6) + \\
 &(2 \times 4) + (1 \times 3) + (2 \times 4) + \\
 &(1 \times 3) + (-2 \times 9) + (0 \times 2) = 5
 \end{aligned}$$

1) Patch Extraction and Representation

$$F_1(\mathbf{Y}) = \max(0, W_1 * \mathbf{Y} + B_1)$$

- W_1 : n_1 filters of support $c \times f_1 \times f_1$
- B_1 : n_1 - dimensional vector (bias)
- $*$: convolution operation
- $\max(0, x)$: ReLU operation (non-linearity)



2) Non-linear mapping

$$F_2(\mathbf{Y}) = \max(0, \boxed{W_2} * F_1(\mathbf{Y}) + B_2)$$

n_2 filters of size $n_1 \times f_2 \times f_2$
($f_2 = 1$)

- W_2 : n_2 filters of support $n_1 \times f_2 \times f_2$
- B_2 : n_2 - dimensional vector (bias)
- $*$: convolution operation
- $\max(0, x)$: ReLU operation (non-linearity)

< 1x1 Convolution Operation >

- Takes the n_1 -dimensional feature vector at location (x, y) from $F_1(\mathbf{Y})$
- It performs a dot product between this n_1 dimensional feature vector and 1x1 filter (which is also conceptually an n_1 dimensional vector, as it spans across all input channels from $F_1(\mathbf{Y})$)

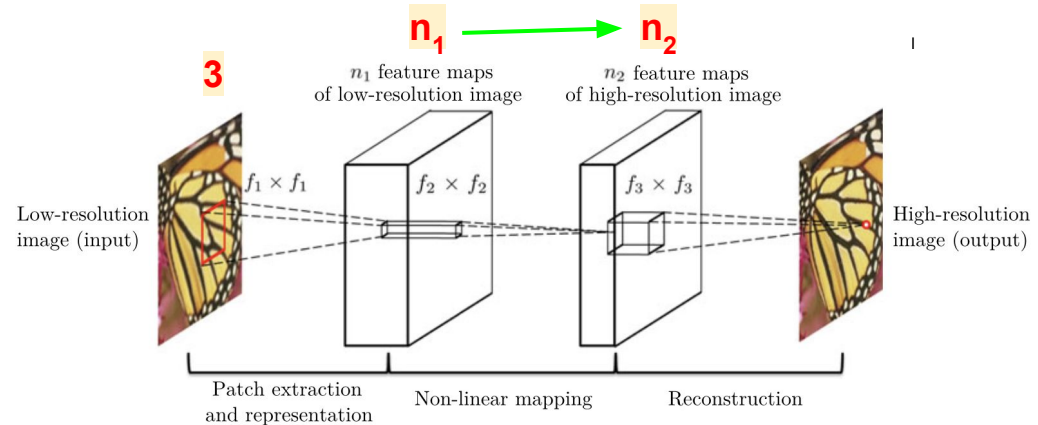
2) Non-linear mapping

$$F_2(\mathbf{Y}) = \max(0, W_2 * F_1(\mathbf{Y}) + B_2)$$

- W_2 : n_2 filters of support $n_1 \times f_2 \times f_2$
- B_2 : n_2 - dimensional vector (bias)
- $*$: convolution operation
- $\max(0, x)$: ReLU operation (non-linearity)

< 1x1 Convolution Operation >

- Takes the n_1 -dimensional feature vector at location (x, y) from $F_1(\mathbf{Y})$
- It performs a dot product between this n_1 dimensional feature vector and 1x1 filter (which is also conceptually an n_1 dimensional vector, as it spans across all input channels from $F_1(\mathbf{Y})$)



3) Reconstruction

$$F(\mathbf{Y}) = \boxed{W_3} * F_2(\mathbf{Y}) + B_3$$

c filters of size $n_2 \times f_3 \times f_3$

- W_3 : c filters of support $n_2 \times f_3 \times f_3$
- B_3 : c- dimensional vector (bias)
- $*$: convolution operation
- $\max(0, x)$: ReLU operation (non-linearity)

Traditional methods use averaging to combine overlapping patches. In this scenario, the filters in W_3 are expected to act like an averaging filter.

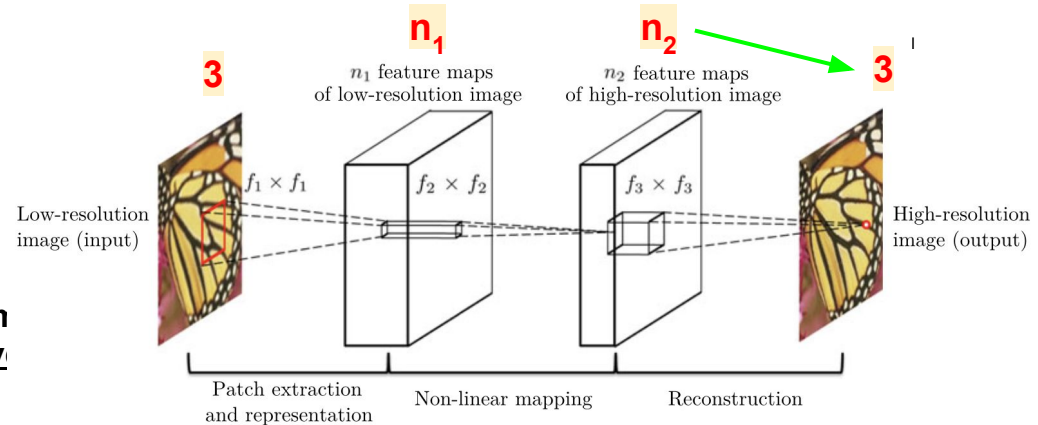
3) Reconstruction

$$F(\mathbf{Y}) = \boxed{W_3} * F_2(\mathbf{Y}) + B_3$$

c filters of size $n_2 \times f_3 \times f_3$

- W_3 : c filters of support $n_2 \times f_3 \times f_3$
- B_3 : c - dimensional vector (bias)
- $*$: convolution operation
- $\max(0, x)$: ReLU operation (non-linearity)

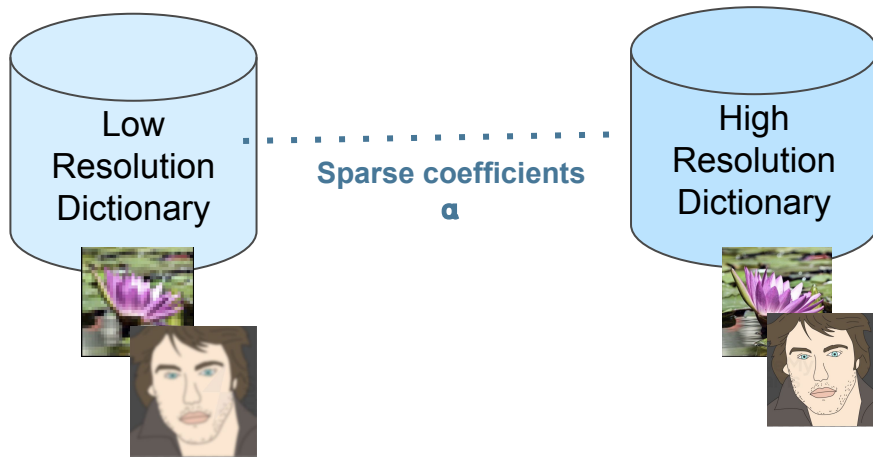
Traditional methods use averaging to con
filters in W_3 are expected to act like an av



Relationship to Sparse-Coding-Based Methods:

Traditional Super Resolution Pipeline

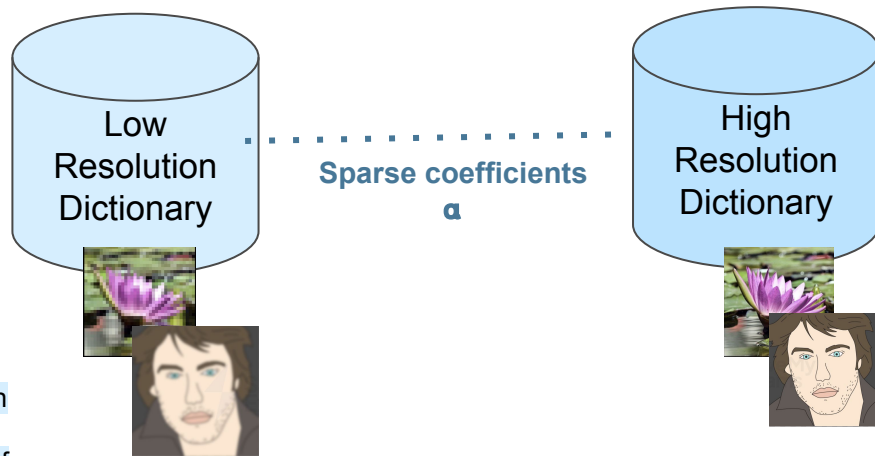
Traditional SR Pipeline rely on “pre-learned” dictionaries



Relationship to Sparse-Coding-Based Methods:

Traditional Super Resolution Pipeline

Traditional SR Pipeline rely on “pre-learned” dictionaries



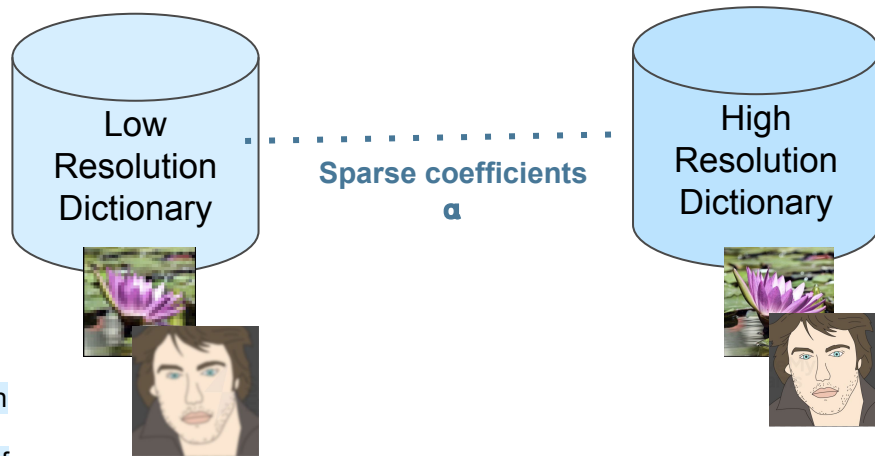
Low-Resolution Dictionary

- Like a vocabulary of common patterns found in low-resolution image patches.
- Learned from a large dataset of low-resolution images

Relationship to Sparse-Coding-Based Methods:

Traditional Super Resolution Pipeline

Traditional SR Pipeline rely on “pre-learned” dictionaries

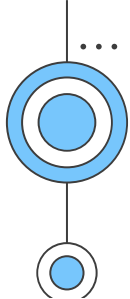


Low-Resolution Dictionary

- Like a vocabulary of common patterns found in low-resolution image patches.
- Learned from a large dataset of low-resolution images

High-Resolution Dictionary

- Like a vocabulary of corresponding patterns found in high-resolution image patches.
- Learned from the same dataset, but using the high-resolution versions of the images

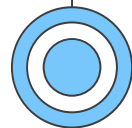
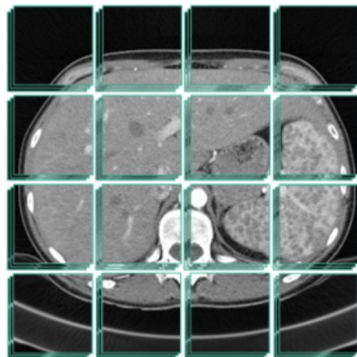


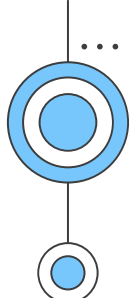
Relationship to Sparse-Coding-Based Methods:

Traditional Super Resolution Pipeline

Step 1) Patch Extraction

: Divide the low resolution input image into overlapping patches





Relationship to Sparse-Coding-Based Methods:

Traditional Super Resolution Pipeline

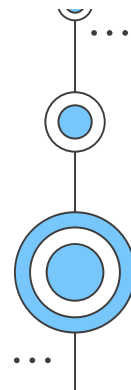
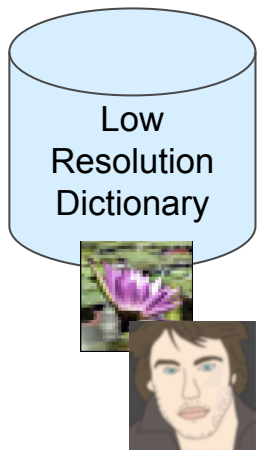
Step 1) Patch Extraction

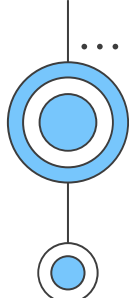
: Divide the low resolution input image into overlapping patches

Step 2) Sparse Coding

: For each low resolution image patch y , find its sparse representation α using a low resolution dictionary D_l

$$y = D_l \alpha$$





Relationship to Sparse-Coding-Based Methods:

Traditional Super Resolution Pipeline

Step 1) Patch Extraction

: Divide the low resolution input image into overlapping patches

Step 2) Sparse Coding

: For each low resolution image patch y , find its sparse representation α using a low resolution dictionary D_l

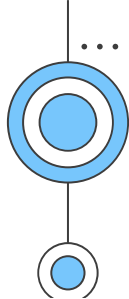
$$y = D_l \alpha$$

Step 3) High-Resolution Reconstruction

: Use the sparse coefficients α and a corresponding high-resolution dictionary D_h to reconstruct a high-resolution patch x

$$X = D_h \alpha$$





Relationship to Sparse-Coding-Based Methods:

Traditional Super Resolution Pipeline

Step 1) Patch Extraction

: Divide the low resolution input image into overlapping patches

Step 2) Sparse Coding

: For each low resolution image patch y , find its sparse representation α using a low resolution dictionary D_l

$$y = D_l \alpha$$

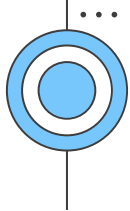
Step 3) High-Resolution Reconstruction

: Use the sparse coefficients α and a corresponding high-resolution dictionary D_h to reconstruct a high-resolution patch x

$$X = D_h \alpha$$

Step 4) Aggregation

: Average the overlapping high-resolution patches to correlate the final high resolution image



Relationship to Sparse-Coding-Based Methods

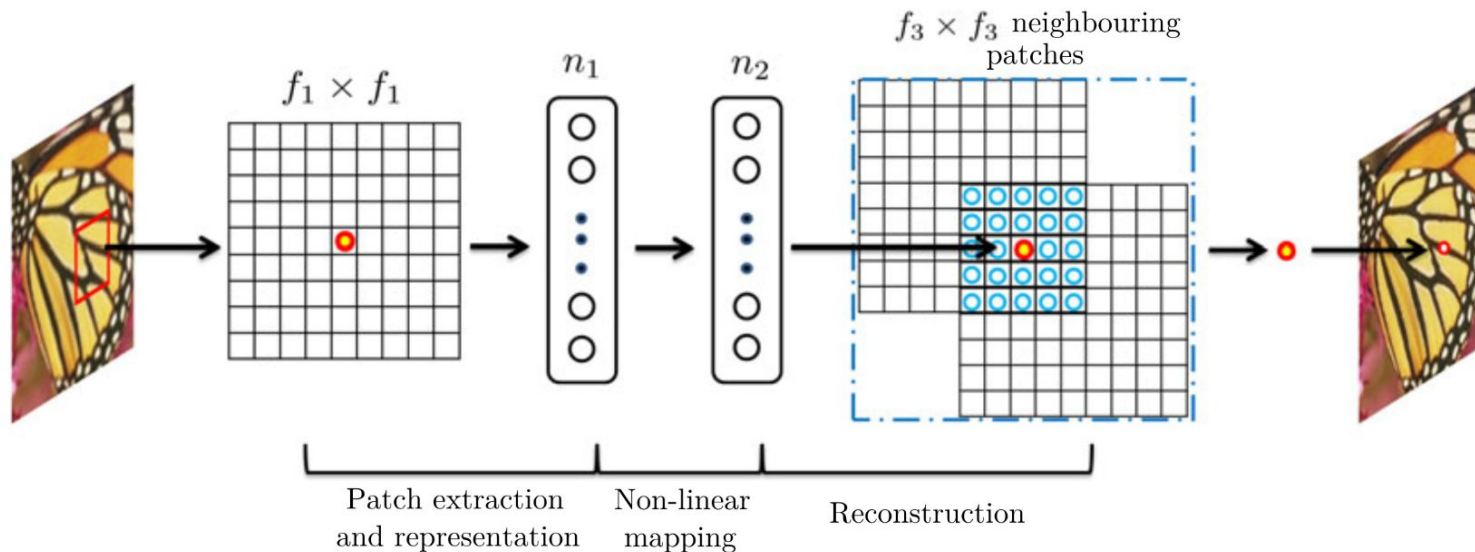
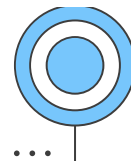
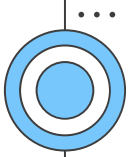
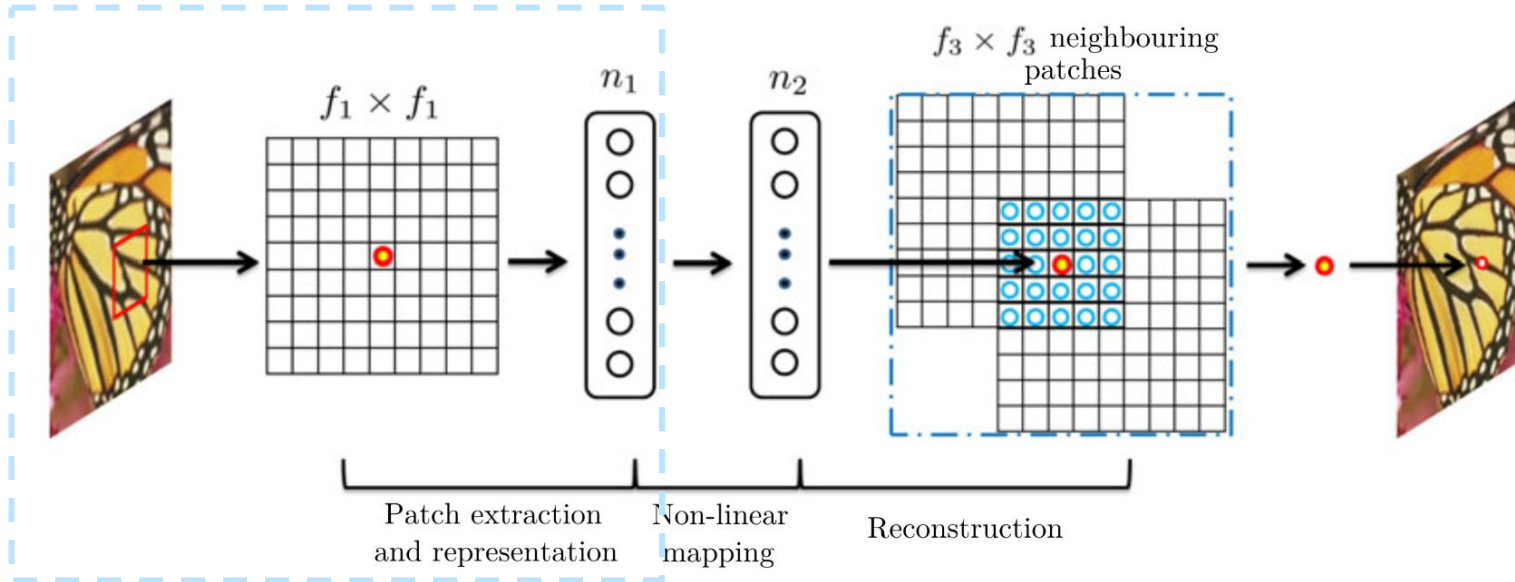


Fig. 3. An illustration of sparse-coding-based methods in the view of a convolutional neural network.





Relationship to Sparse-Coding-Based Methods



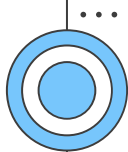
[Sparse-Coding Based Methods]

Step 1) Patch Extraction

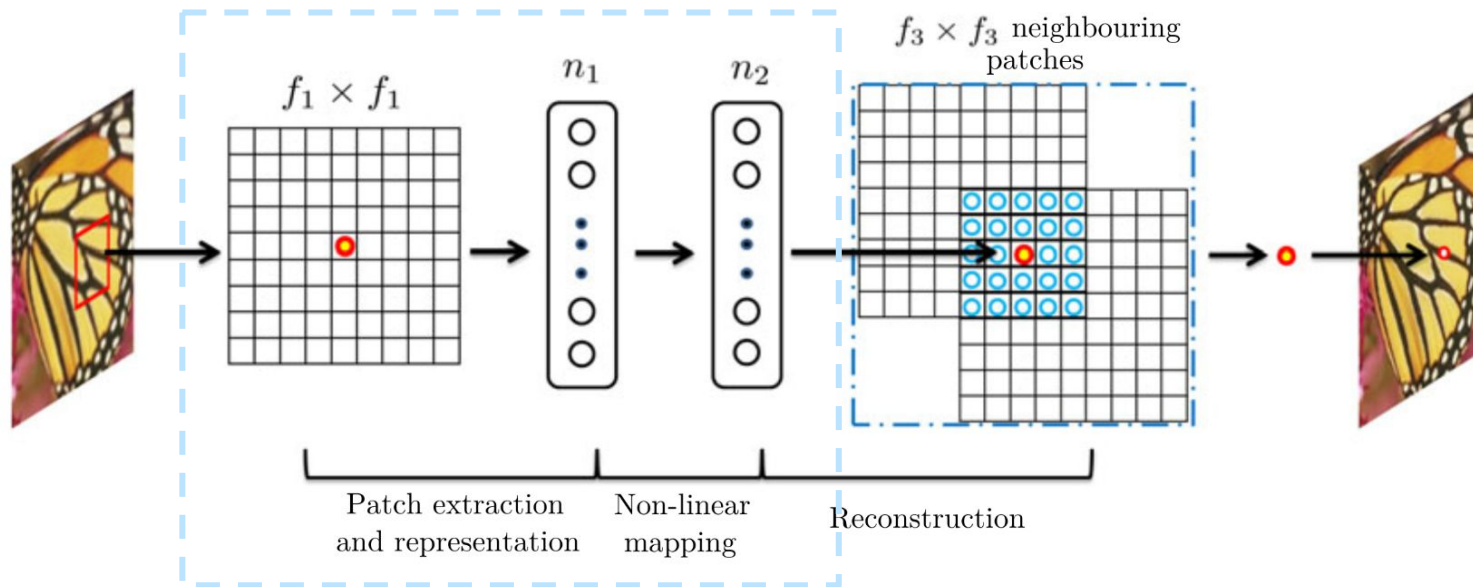
Step 2) Sparse Coding

Step 3) High-Resolution Reconstruction

Step 4) Aggregation



Relationship to Sparse-Coding-Based Methods



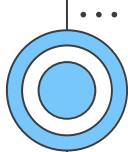
[Sparse-Coding Based Methods]

Step 1) Patch Extraction

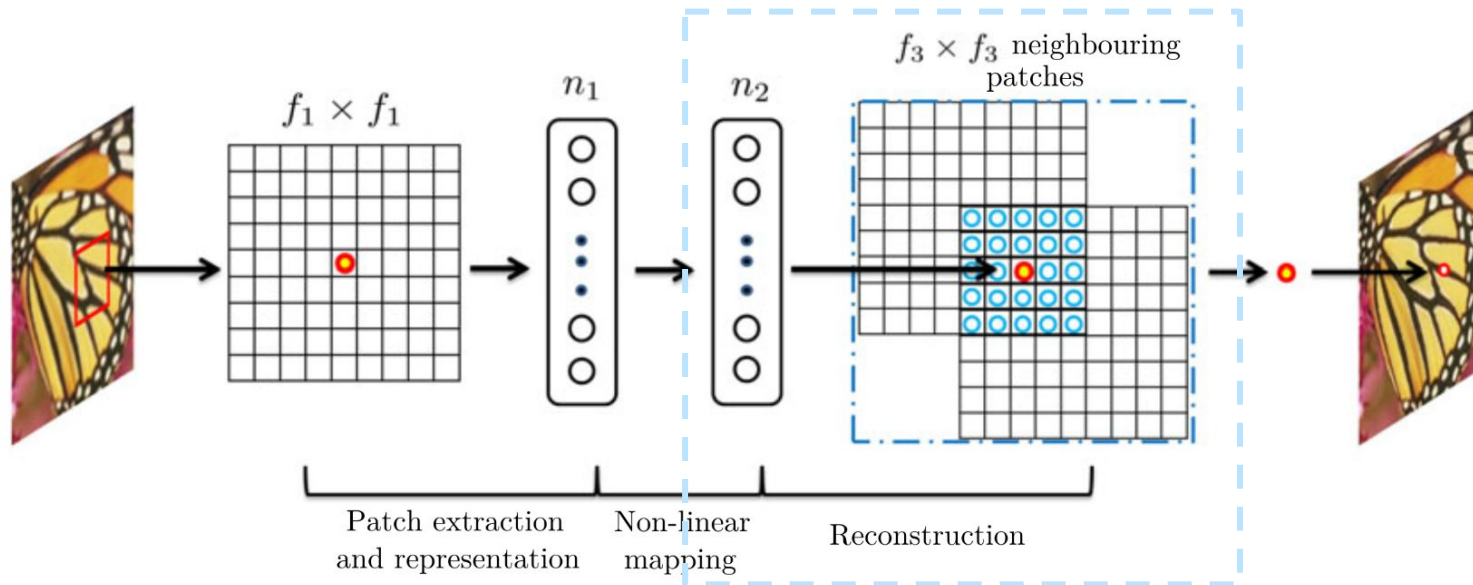
Step 2) Sparse Coding

Step 3) High-Resolution Reconstruction

Step 4) Aggregation



Relationship to Sparse-Coding-Based Methods



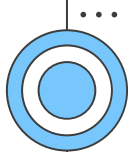
[Sparse-Coding Based Methods]

Step 1) Patch Extraction

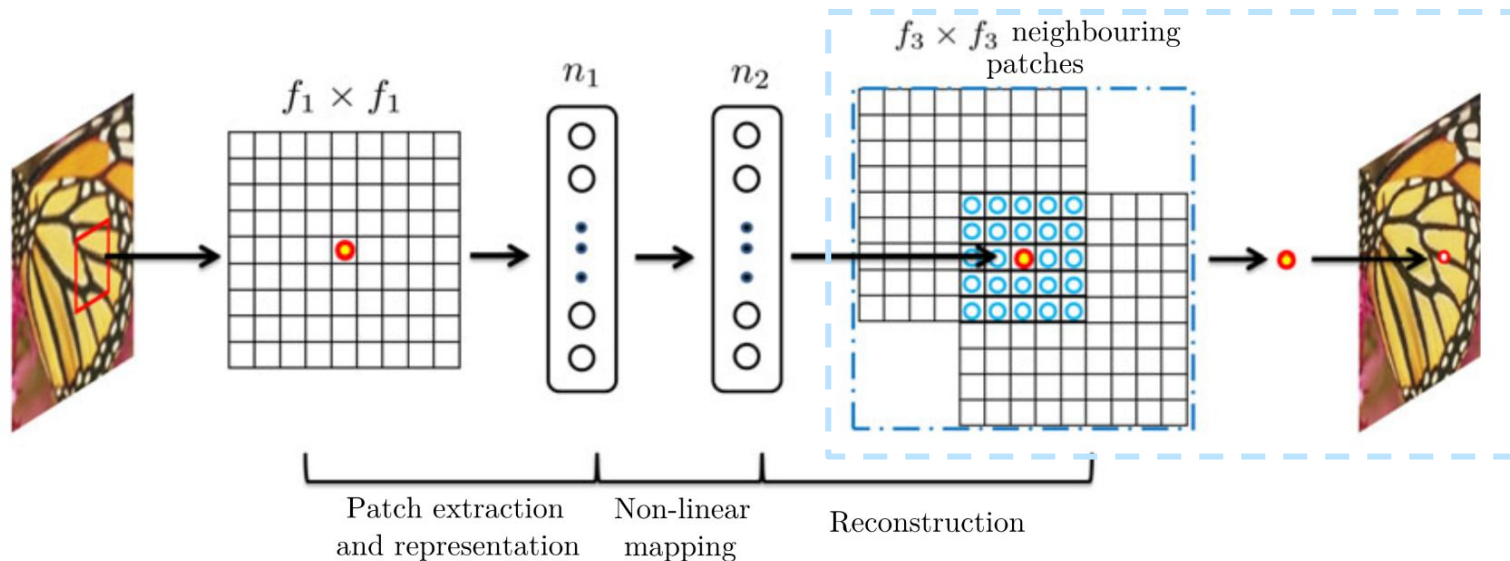
Step 2) Sparse Coding

Step 3) High-Resolution Reconstruction

Step 4) Aggregation



Relationship to Sparse-Coding-Based Methods



[Sparse-Coding Based Methods]

Step 1) Patch Extraction

Step 2) Sparse Coding

Step 3) High-Resolution Reconstruction

Step 4) Aggregation

Training

Loss function:

$$L(\Theta) = \frac{1}{n} \sum_{i=1}^n \|F(\mathbf{Y}_i; \Theta) - \mathbf{X}_i\|^2$$

Θ : $\{W_1, W_2, W_3, B_1, B_2, B_3\}$

$\{\mathbf{X}_i\}$: A set of high-resolution images

$\{\mathbf{Y}_i\}$: A set of the corresponding low-resolution images

Training

Loss function:

$$L(\Theta) = \frac{1}{n} \sum_{i=1}^n ||F(\mathbf{Y}_i; \Theta) - \mathbf{X}_i||^2$$

Θ : $\{W_1, W_2, W_3, B_1, B_2, B_3\}$

$\{X_i\}$: A set of high-resolution images

$\{Y_i\}$: A set of the corresponding low-resolution images

Optimization:

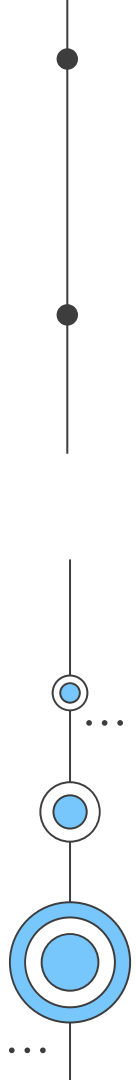
$$\Delta_{i+1} = 0.9 \cdot \Delta_i + \eta \cdot \frac{\partial L}{\partial W_i^\ell}, \quad W_{i+1}^\ell = W_i^\ell + \Delta_{i+1}$$

Using the Stochastic Gradient Descent



03

Experiments



Experiments: w.r.t the dataset size

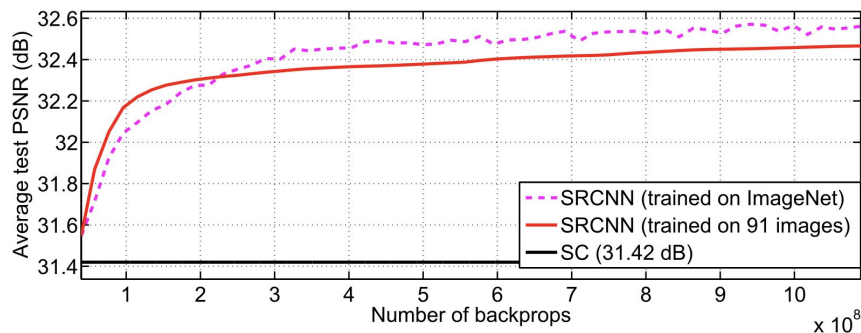
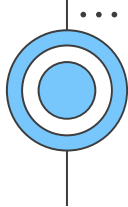


Fig. 4. Training with the much larger ImageNet dataset improves the performance over the use of 91 images.

The results positively indicate that SRCNN performance may be **further boosted using a larger training set**, but the effect of big data is **not as impressive** as that shown in high-level vision problems. This is mainly because that the 91 images have already captured sufficient variability of natural images.



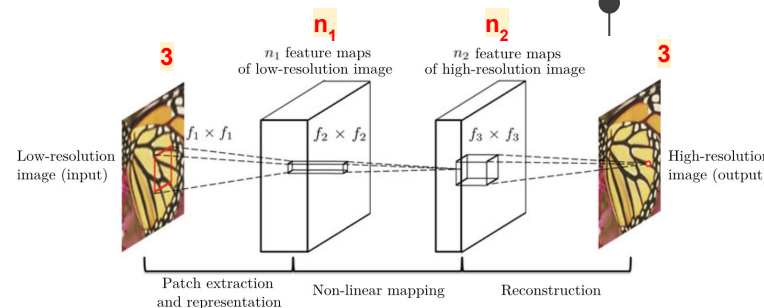
Experiments: w.r.t the filter numbers

TABLE 1

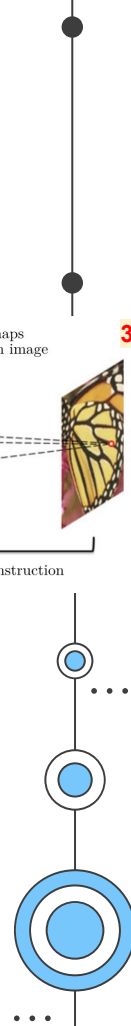
The Results of Using Different Filter Numbers in SRCNN

$n_1 = 128$ $n_2 = 64$		$n_1 = 64$ $n_2 = 32$		$n_1 = 32$ $n_2 = 16$	
PSNR	Time (sec)	PSNR	Time (sec)	PSNR	Time (sec)
32.60	0.60	32.52	0.18	32.26	0.05

Training is performed on ImageNet whilst the evaluation is conducted on the Set5 dataset.



It is clear that **superior performance** could be achieved by **increasing the width**. However, if a fast restoration speed is desired, a small network width is preferred, which could still achieve better performance than the sparse-coding-based method.



Experiments: comparison with other methods

TABLE 2
The Average Results of PSNR (dB), SSIM, IFC, NQM, WPSNR (dB) and MSSIM on the Set5 Dataset

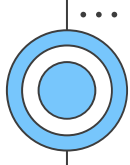
Eval. Mat	Scale	Bicubic	SC [48]	NE+LLE [4]	KK [24]	ANR [39]	A+ [39]	SRCNN
PSNR	2	33.66	-	35.77	36.20	35.83	36.54	36.66
	3	30.39	31.42	31.84	32.28	31.92	32.59	32.75
	4	28.42	-	29.61	30.03	29.69	30.28	30.49
SSIM	2	0.9299	-	0.9490	0.9511	0.9499	0.9544	0.9542
	3	0.8682	0.8821	0.8956	0.9033	0.8968	0.9088	0.9090
	4	0.8104	-	0.8402	0.8541	0.8419	0.8603	0.8628

TABLE 3
The Average Results of PSNR (dB), SSIM, IFC, NQM, WPSNR (dB) and MSSIM on the Set14 Dataset

Eval. Mat	Scale	Bicubic	SC [48]	NE+LLE [4]	KK [24]	ANR [39]	A+ [39]	SRCNN
PSNR	2	30.23	-	31.76	32.11	31.80	32.28	32.45
	3	27.54	28.31	28.60	28.94	28.65	29.13	29.30
	4	26.00	-	26.81	27.14	26.85	27.32	27.50
SSIM	2	0.8687	-	0.8993	0.9026	0.9004	0.9056	0.9067
	3	0.7736	0.7954	0.8076	0.8132	0.8093	0.8188	0.8215
	4	0.7019	-	0.7331	0.7419	0.7352	0.7491	0.7513

TABLE 4
The Average Results of PSNR (dB), SSIM, IFC, NQM, WPSNR (dB) and MSSIM on the BSD200 Dataset

Eval. Mat	Scale	Bicubic	SC [48]	NE+LLE [4]	KK [24]	ANR [39]	A+ [39]	SRCNN
PSNR	2	28.38	-	29.67	30.02	29.72	30.14	30.29
	3	25.94	26.54	26.67	26.89	26.72	27.05	27.18
	4	24.65	-	25.21	25.38	25.25	25.51	25.60
SSIM	2	0.8524	-	0.8886	0.8935	0.8900	0.8966	0.8977
	3	0.7469	0.7729	0.7823	0.7881	0.7843	0.7945	0.7971
	4	0.6727	-	0.7037	0.7093	0.7060	0.7171	0.7184



Experiments: Qualitative Analysis

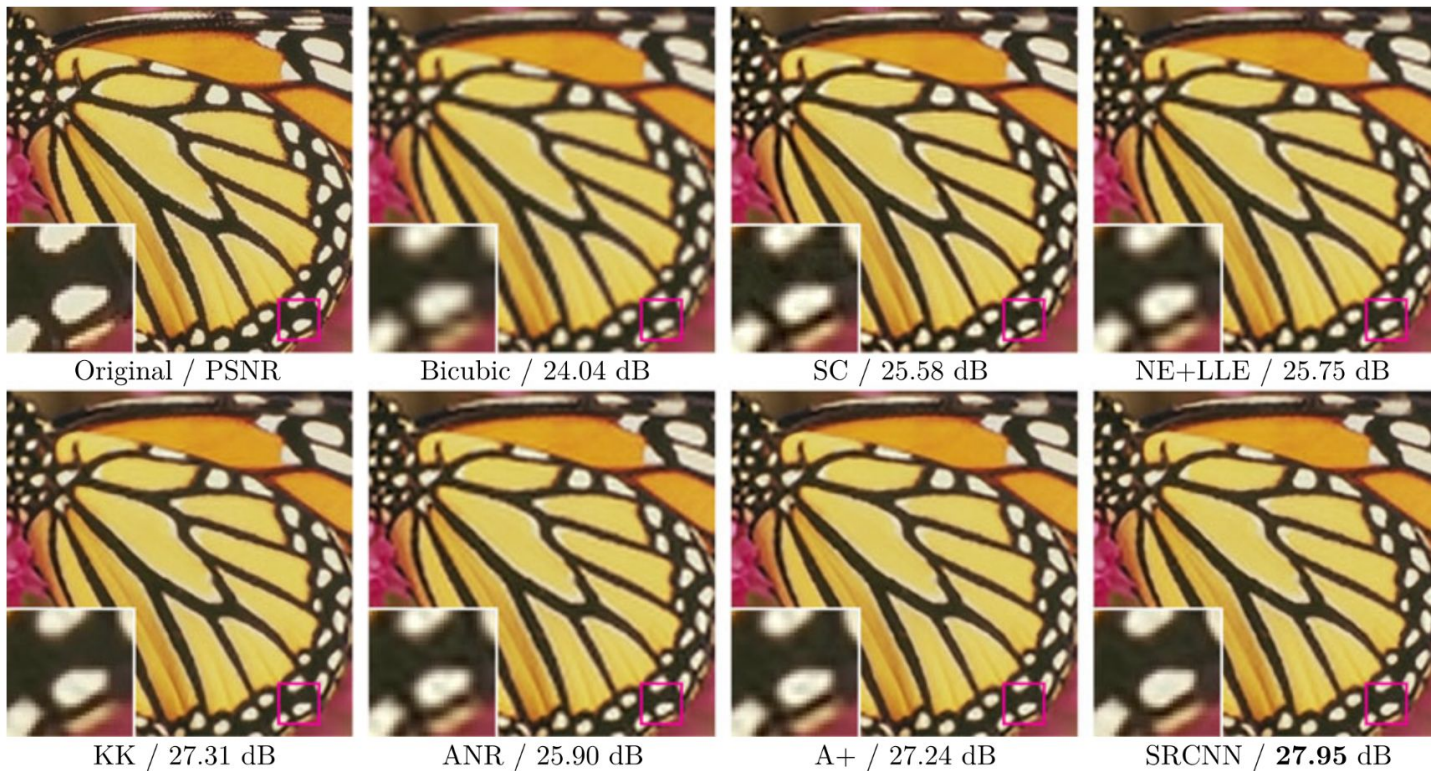
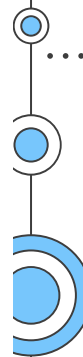
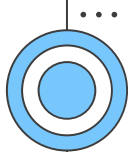
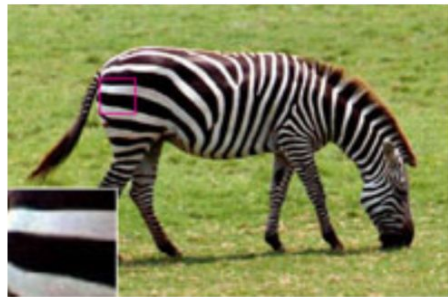


Fig. 14. The “butterfly” image from Set5 with an upscaling factor 3.

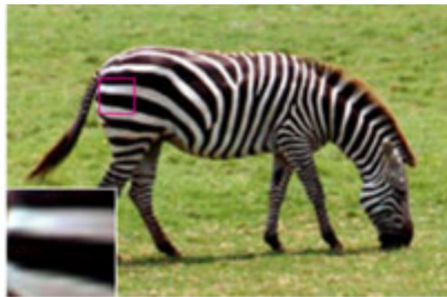




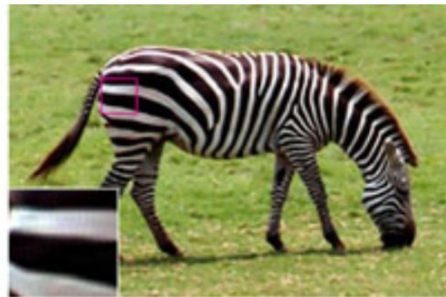
Experiments: Qualitative Analysis



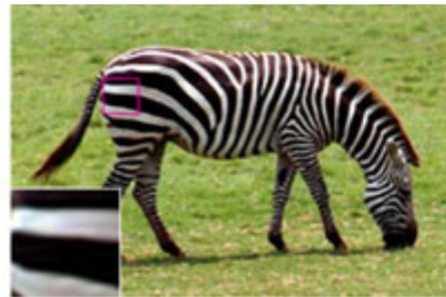
Original / PSNR



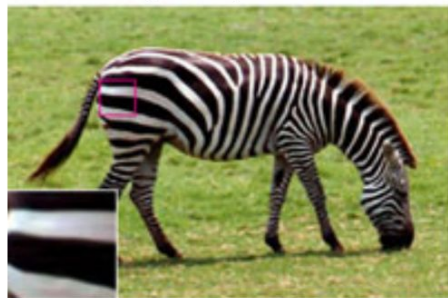
Bicubic / 26.63 dB



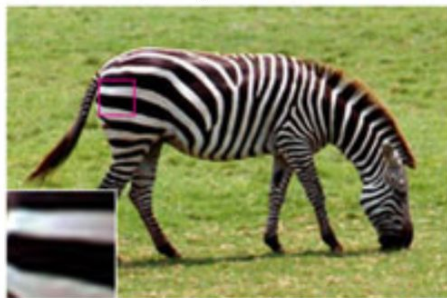
SC / 27.95 dB



NE+LLE / 28.31 dB



KK / 28.85 dB



ANR / 28.43 dB

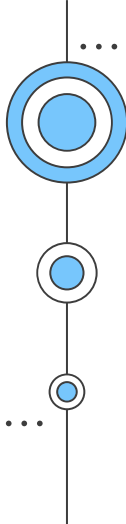


A+ / 28.98 dB



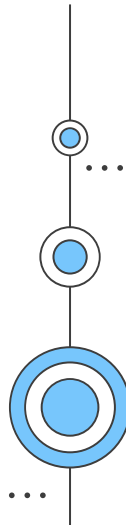
SRCNN / **29.29 dB**





04

Conclusion



Conclusion

- **Showed that conventional sparse-coding based SR methods can be reformulated into a deep convolutional neural network**
- **Proposed SRCNN model that:**
 - Needs little extra pre/post processing
 - Is lightweight
 - Has achieved superior performance than the other state-of-the-art methods